Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 2601058
Name: Vedang Pravin Gadge

CODE:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List (append at end)
    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    # 2. Traverse and print
    def traverse(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

    # 3. Insert at specific position
    def insert_at(self, pos, data):
        new_node = Node(data)
```

```
                if pos == 0:
            new_node.next = self.head
            self.head = new_node
            return
        temp = self.head
        for _ in range(pos-1):
            if temp is None:
                print("Position out of range")
                return
            temp = temp.next
        new_node.next = temp.next
        temp.next = new_node

    # 4. Find Middle node
    def find_middle(self):
        slow = self.head
        fast = self.head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        if slow:
            print("Middle node value:", slow.data)

    # 5. Delete node (by value)
    def delete(self, key):
        temp = self.head
        if temp and temp.data == key:
            self.head = temp.next
            return
        prev = None
        while temp and temp.data != key:
            prev = temp
            temp = temp.next
        if temp:
            prev.next = temp.next

    # 6. Reverse list
    def reverse(self):
        prev = None
        curr = self.head
        while curr:
            nxt = curr.next
            curr.next = prev
```

```
                prev = curr
            curr = nxt
        self.head = prev

    # 7. Sum of every two consecutive nodes
    def sum_consecutive_pairs(self):
        temp = self.head
        while temp and temp.next:
            print(f"Sum({temp.data}, {temp.next.data}) = {temp.data + temp.next.data}")
            temp = temp.next

ll = LinkedList()
ll.append(10)
ll.append(20)
ll.append(30)
ll.append(40)
ll.append(50)

print("Original List:")
ll.traverse()

ll.insert_at(2, 99)
print("\nAfter inserting 99 at position 2:")
ll.traverse()

ll.find_middle()

ll.delete(30)
print("\nAfter deleting 30:")
ll.traverse()

ll.reverse()
print("\nAfter reversing:")
ll.traverse()

print("\nSum of consecutive pairs:")
ll.sum_consecutive_pairs()
```

SCREENSHOT:

```
Original List:
10 -> 20 -> 30 -> 40 -> 50 -> None

After inserting 99 at position 2:
10 -> 20 -> 99 -> 30 -> 40 -> 50 -> None
Middle node value: 30

After deleting 30:
10 -> 20 -> 99 -> 40 -> 50 -> None

After reversing:
50 -> 40 -> 99 -> 20 -> 10 -> None

Sum of consecutive pairs:
Sum(50, 40) = 90
Sum(40, 99) = 139
Sum(99, 20) = 119
Sum(20, 10) = 30
```